ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/220642519

Many-Valued Logic and Mixed Integer Programming.

Article *in* Annals of Mathematics and Artificial Intelligence · December 1994

DOI: 10.1007/BF01530787 · Source: DBLP

CITATIONS
115

READS
245

1 author:

Reiner Hähnle
Technische Universität Darmstadt

238 PUBLICATIONS **5,228** CITATIONS

SEE PROFILE

All content following this page was uploaded by Reiner Hähnle on 16 May 2014.

The user has requested enhancement of the downloaded file.

Many-Valued Logic and Mixed Integer Programming

Reiner Hähnle*

Institut für Logik, Komplexität und Deduktionssysteme
Fakultät für Informatik, Universität Karlsruhe
76128 Karlsruhe, Germany
e-mail: haehnle@ira.uka.de

Abstract. We generalize propositional semantic tableaux for classical and many-valued logics to *constraint tableaux*. We show that this technique is a generalization of the standard translation from CNF formulas into integer programming. The main advantages are (i) a relatively efficient satisfiability checking procedure for classical, finitely-valued and, for the first time, for a wide range of infinitely-valued propositional logics; (ii) easy NP-containment proofs for many-valued logics. The standard translation of two-valued CNF formulas into integer programs and Tseitin's structure preserving clause form translation are obtained as a special case of our approach.

Keywords: Many-Valued Logic, Mechanical Theorem Proving, Integer Programming, Linear Programming, Constraint Programming

Introduction

Following the pioneering work done by Jeroslow [27–29], there has been considerable interest recently in the interaction of methodologies from artificial intelligence (AI) and operations research (OR), more specifically, between symbolic satisfiability checking in logic and (mixed) integer programming [23–26, 22, 30, 4, 2, 44, 5]. The research efforts in this area can be roughly divided into three groups. The first group of papers [29, 23, 22, 24] is mainly concerned about improvement of satisfiability checking algorithms for twovalued propositional logic by combining insights from AI and OR. The second group [27, 30, 26, 44] investigates the use of integer programming methods in first-order automated theorem proving. The present paper belongs to the third group [4, 2, 5] that is interested in using OR methods to *extend* two-valued logic in various ways. While the first two groups of papers have an improvement in performance as their ultimate goal, the second and, even more so, the third group is also interested in a *uniform treatment* of a broad range of problems of a possibly hybrid nature.

* Part of the reserach reported here was carried out while the author was supported by a grant within the DFG Schwerpunktprogramm Deduktion. Preliminary and partial versions of this paper were published as [15, 16].

In this paper we give a translation of deduction problems from many-valued propositional logic to mixed integer programming (MIP) problems. We show that this translation generalizes the standard translation [11, 31] of two-valued propositional formulas in conjunctive normal form (CNF) to 0-1 integer programs. The generalization is twofold: first, it applies to all finitely-valued and some infinitely-valued logics and second, it does not presuppose a normal form as its input. The latter requirement is important in many-valued logics, since for most of them no natural normal forms are known.

As the standard translation in the two-valued CNF case, our general translation provides a reduction of satisfiability in many-valued logics to solvability of MIP problems for a very broad class of many-valued logics, including all finitely-valued logics. From this we draw two immediate consequences:

1. We have a uniform and relatively fast decision procedure for satisfiability in these logics, namely any general MIP implementation.
2. Since MIP$\in$NP, we have the same result for all logics where our translation is in polynomial time. These logics include most infinitely-valued logics in the literature.[1]

Recently, interesting applications for many-valued theorem proving on the propositional level in hardware verification [21], respectively, in adaptive error-correcting codes [37] have been suggested. With the techniques presented in this paper it becomes possible to handle problems that go far beyond text book examples. While the present paper is only concerned with propositional logic, results and techniques as developed in [30, 26, 44] are orthogonal to the many-valued issues addressed in the following. Therefore, the results of the present paper could be combined with those on classical first-order logic and integer programming techniques to solve many-valued first-order problems which arise, for example, in program verification [1, 8, 6].

The paper is organized as follows: in Section 1 we state some background material on theorem proving with analytic tableaux and many-valued logics. In Section 2 we define a generalization of many-valued analytic tableaux which will be illustrated by an example in Section 3. We state a reduction from two-valued logic to IP in Section 4. In Section 5 we show how many-valued logics in can be reduced to MIP problems. In Section 6 we point out how our translation generalizes earlier the results by Dantzig and Karp and we show some connections to algorithms for computation of short CNF in two-valued logic. Finally, we lay down our future research goals.

1 Prerequisites

1.1 Mixed Integer Programming

A general MIP problem consists of minimizing a linear function with respect to a set of constraints consisting of linear inequations in which real and integer variables can occur [45], more precisely:

[1] For finitely-valued logics the result is trivial.

Definition 1 (MIP). *Let $\boldsymbol{x} = (x_1, \ldots, x_k)$ and $\boldsymbol{y} = (y_1, \ldots, y_m)$ be variables over the reals, respectively, over the integers and let A, B be integer matrices and h an integer vector. Let $f(\boldsymbol{x}, \boldsymbol{y})$ be an $k+m$-place linear function. Then a* **general MIP problem** *is to find $\underline{\boldsymbol{x}} \in \mathbb{R}^k$, $\underline{\boldsymbol{y}} \in \mathbb{Z}^m$ such that*

$$f(\underline{\boldsymbol{x}}, \underline{\boldsymbol{y}}) = \min\{f(\boldsymbol{x}, \boldsymbol{y}) |\ A\boldsymbol{x} + B\boldsymbol{y} \geq h\}$$

The general case can be restricted for the concerns of this paper. We deal with *bounded* MIP (bMIP) problems, more precisely, all solutions will be in the interval $[0, 1]$. Moreover, it is not necessary to leave the domain of the rational numbers while solving an MIP problem, so we can assume that $\boldsymbol{x} \in \mathbb{Q}^k$. If there are no rational variables present we have a 0-1 integer programming (IP) problem. From now we denote with $[i, j]$ the closed rational interval ranging from i to j.

Proposition 1. *bMIP is NP-complete.*

Proof. bMIP is NP-hard, since already 0-1 IP is NP-hard [31]. On the other hand, bMIP∈NP, since it suffices to guess the 0-1 vector $\underline{\boldsymbol{y}}$ and solve the resulting linear program in P time.

Definition 2 (bMIP-representable [28]). *A set $M \subseteq [0, 1]^k$ is* **bMIP-representable** *iff there is a bMIP (A, B, h) with k real and m 0-1 variables such that*

$$M = \{\boldsymbol{x} \mid \textit{there is } \boldsymbol{y} \in \{0, 1\}^m \textit{ such that } A\boldsymbol{x} + B\boldsymbol{y} \geq h\}$$

It will be convenient to stretch usual terminology a bit. A bounded IP (bIP) problem normally is an MIP problem where all variables are over some interval $[-a, b] \subseteq \mathbb{Z}$. In the following we understand that a bIP problem has solutions in a finite set of rational numbers of the form $\left\{0, \frac{1}{n-1}, \ldots, \frac{n-2}{n-1}, 1\right\}$. It is trivial to transform such bIP problems into equivalent ones in the usual sense.

For an introductions to IP research, see [28, 45].[2]

1.2 Many-Valued Logic

Definition 3 (Syntax, Truth Values). *Let L be a propositional language with propositional variables L_0 and connectives $\mathbf{F}$. Let N be the set of* **truth values**. *N is either finite and consists of equidistant rational numbers between 0 and 1 in which case $N = \left\{0, \frac{1}{n-1}, \ldots, \frac{n-2}{n-1}, 1\right\}$ or it is infinite and consists of the rational interval $[0, 1]$. If N is finite we denote with n the cardinality of N.*

[2] In the first work also some connections to automated deduction in classical logics are mentioned.

Definition 4 (Semantics, Many-Valued Logic). *k-ary connectives $F \in \mathbf{F}$ are interpreted as functions $f : N^k \to N$. We call f the* **interpretation** *of F. Let $\mathbf{f}$ be the family of functions associated with connectives in $\mathbf{F}$. Then we call $\mathbf{f}$ n-valued matrix for L and the triple $\langle L, \mathbf{f}, N\rangle$* **many-valued propositional logic**.

Definition 5 (Valuation). *Let $\mathcal{L} = \langle L, \mathbf{f}, N\rangle$ be a many-valued propositional logic. A* **valuation** *for $\mathcal{L}$ is a function $v : L_0 \to N$. As usual, v can be uniquely extended to L via*

$$v(F(\phi_1, \ldots, \phi_k)) = f(v(\phi_1), \ldots, v(\phi_k))$$

where f is the interpretation of F.

Definition 6 (S-Satisfiable, S-Tautology). *For $S \subseteq N$ and a many-valued propositional logic $\mathcal{L}$ call a formula $\phi \in L$ S-***satisfiable** *iff there is a valuation such that $v(\phi) \in S$. Call ϕ an S-***tautology** *iff $v(\phi) \in S$ for all valuations v.*

Remark 1. Our notion of many-valued logic is based on satisfiability of (sets of) formulas. There is another, more general, notion of many-valued logic based on consequence relations [49] which is, however, not suitable for automated theorem proving. If we must deal with logical consequence, then we assume that it can be expressed on the object level via a suitable deduction theorem (cf. [42, 10]).

The general satisfiability problem in many-valued logics is to determine, whether for a given set of formulas $\{\phi_1, \ldots, \phi_k\}$ and given sets of truth values $S_1, \ldots, S_k \subseteq N$ all ϕ_i are simultaneously S_i-satisfiable for $1 \leq i \leq k$ in a given logic. We do not allow arbitrary choices of the S_i and arbitrary interpretations of connectives. We impose the following restrictions which are justified by the fact that virtually all many-valued logics investigated in the literature are subsumed by it.

1. We restrict queries for S-satisfiability of formulas to sets of the form

$$S = ([i_1, j_1] \cup \cdots \cup [i_m, j_m]) \cap N$$

2. For each k-ary connective f the set $\mathrm{graph}(f) = \{(i_1, \ldots, i_k, j) | f(i_1, \ldots, i_k) = j\}$ must be bMIP-representable.

For the rest of the paper we assume these restrictions (note that finitely-valued logics are not restricted at all by these assumptions). Moreover, we assume from now on that all connectives are at most binary.[3]

[3] This is no severe restriction. All results in principle are independent of the arity of the connectives, however, the assumption that they are at most binary makes the presentation much more readable.

1.3 Analytic Tableaux

In this subsection all logics are finitely-valued.

Analytic (or semantic) tableaux are a sound and complete proof procedure for classical first-order predicate logic introduced independently by Hintikka and Beth in the 50s (a standard reference is [47]). They are closely related to Gentzen systems. Analytic tableau systems exist for a wide variety of logics, including order-sorted, modal, intuitionistic, substructural, and temporal logics. The version we present here is modified for the efficient treatment of many-valued logics, cf. [13, 17].

Definition 7 (Signed Formula). *Let $\phi \in L$, $S \subseteq N$. Then we call the expression $S\ \phi$* **signed formula**. *The set of all signed formulas is denoted with L^*.*

Definition 8 (Satisfiability of Signed Formulas). *$S\ \phi \in L^*$ is* **satisfiable** *iff ϕ is S-satisfiable.*

Signed formulas are a device for talking about many-valued logics within classical two-valued logic, the logic of the meta-level.

Analytic tableaux are a refutation procedure. For our purposes it is sufficient to visualize a tableau proof as a finite labelled tree, whose node labels are signed formulas. In order to establish that a formula ϕ is an S-tautology we start with a tableau whose single node is labelled with the complement of S, namely $(N - S)\ \phi$. Now this formula is analysed following its syntactic structure in a top-down manner to the atomic level. If we arrive at a contradiction in any case we have proved that no valuation can satisfy the root, in other words, ϕ is a S-tautology. Before we give the formal definitions we illustrate the process with a simple example from classical logic.

Example 1. We prove that $(p \supset \neg p) \supset \neg p$ is a classical tautology, that is, $\{1\}\ (p \supset \neg p) \supset \neg p$ is satisfied by all valuations. We begin by writing down its complement:

$$\{0\}\ (p \supset \neg p) \supset \neg p$$

Next, we analyze the truth conditions for $\{0\}$ and the top level connective $\supset$. For each combination of sign and connective there is a rule that characterizes it. In the present case we need the following rules:

$$\frac{\{1\}\ \phi \supset \psi}{\{0\}\ \phi \,\Big|\, \{1\}\ \psi} \qquad\qquad \frac{\{0\}\ \phi \supset \psi}{\begin{array}{c}\{1\}\ \phi \\ \{0\}\ \psi\end{array}}$$

In our example (see Figure 1) we apply first the rule on the right and then to the first of the resulting formulas the rule on the left. Formulas within the same branch are conjunctively connected while formulas in different branches are disjunctively connected. We notice that each branch in the example contains a **complementary pair** of formulas, that is, $S_1\ \phi$, $S_2\ \phi$ with $S_1 \cap S_2 = \emptyset$. Such branches are called **closed**.

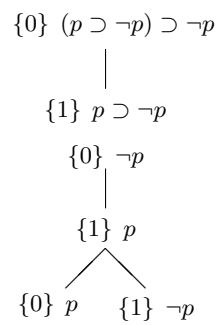

Fig. 1. A tableau proof in two-valued propositional logic.

Definition 9 (Tableau Rule). *A* **tableau rule** *is a tuple* $R{=}\langle \phi, r, \Phi_1, \ldots, \Phi_r \rangle$, *where* $\phi \in L^*$, r *is a positive integer, and* $\Phi_i \in (L^* \dot{\cup} (L^* \times L^*))$. *We define selector functions* $Prem(R) := \phi$, $\alpha(R) := r$, *and* $Ext_i(R) := \Phi_i$ *for the premise, the branching degree, and the* i*th extension of the conclusion of a rule, respectively.*

We say that a tableau rule is **sound** *iff for all valuations* v, *if* v *satisfies the premise, then* v *satisfies at least one extension. We say that a tableau rule is* **complete** *iff for all valuations* v, *if* v *satisfies at least one extension, then* v *satisfies the premise.*

From now on we always work with a fixed, finite set of tableau rules.

Definition 10 (Tableau). *A* **tableau** *for a finite set of signed formulas* Φ *is a finite labelled tree constructed according to the following two rules:*

1. *A finite, linear tree whose nodes are labelled with elements from* Φ *is a tableau for* Φ.
2. *If* $\mathbf{T}$ *is tableau for* Φ, ϕ *is a label occurring on a branch* $\boldsymbol{B}$ *of* T, *and there is a sound and complete tableau rule* R *and a matching substitution* $\sigma : L^* \to L^*$ *such that* $Prem(R)\sigma = \phi$, *then append* $\alpha(R)$ *new linear subtrees to* $\boldsymbol{B}$ *that contain the labels occurring in* $Ext_1(R)\sigma, \ldots, Ext_{\alpha(R)}(R)\sigma$, *respectively.*

The extension of this framework to many-valued logics is completely straightforward. To prove that ϕ is a S-tautology we simply construct a many-valued tableau for $(N - S)\ \phi$. Many-valued tableau rules can be stated very much like their two-valued counterparts. For instance, if we define many-valued conjunction as $i \wedge j = \min(i, j)$, where min is the natural minimum on N, we find the following rule for $\{0, \frac{1}{2}\}\ \phi \wedge \psi$ in three-valued logic:

$$\frac{\{0, \frac{1}{2}\}\ \phi \wedge \psi}{\{0, \frac{1}{2}\}\ \phi \,\Big|\, \{0, \frac{1}{2}\}\ \psi}$$

One difference between the two-valued and the many-valued case is that in the latter more than two extensions in the rules may become necessary. Another important difference is the slightly more general notion of branch closure:

Definition 11 (Many-Valued Closure). *A branch in a many-valued tableau is* **closed** *iff (i) either it contains signed formulas* $S_1\,\phi, \ldots, S_m\,\phi$ *such that* $S_1 \cap \cdots \cap S_m = \emptyset$ *or (ii) a single signed formula* $S\,F(\phi_1, \ldots, \phi_k)$ *such that* $rg(f) \cap S = \emptyset$*, where* $rg(f) = \{f(j_1, \ldots, j_k) |\, j_1, \ldots, j_k \in N\}$*. A branch that cannot be closed is called* **open**.

Definition 12 (Completed Tableau). *A tableau is* **completed***, if for all branches* $\boldsymbol{B}$*, either* $\boldsymbol{B}$ *is closed or all possible rule applications to formulas on* $\boldsymbol{B}$ *have been done, that is, no rule application to a formula on* $\boldsymbol{B}$ *produces a formula which is not already present on* $\boldsymbol{B}$*.*

Obviously, every completed tableau for a finite set of signed formulas is finite.

In general, the number m of formulas in clause (i) of Definition 11 that have to be checked for consistency is greater than two as can be seen from the set $\{0, \frac{1}{2}\}\,\phi, \{0, 1\}\,\phi, \{\frac{1}{2}\}\,\phi\}$ in three-valued logic. In [14] a class of logics, so-called *regular logics*, was introduced where $m = 2$ is sufficient and, moreover, all rules are at most binary branching and clause (ii) of Definition 11 does never occur except for logical constants. Then, of course, one has the standard notion of tableau closure. Case (ii) in the definition above captures **self-contradictory** signed formulas, such as $\{1\}\,\perp$ (where $\perp\, \equiv 0$) in two-valued logic.

On the other hand, for some important classes of many-valued logics, in particular for Łukasiewicz logics (which are defined below), this approach is not completely satisfactory, because the branching factor of rules can become as big as the number of truth values [17]. Moreover, the method does not extend to infinitely-valued logics.

For this reason, we develop in the following section the concept of a constraint tableau rule which is then used to construct a compact tableau system for Łukasiewicz (and many other) logics.

Definition 13 (Tableau Proof). *A completed tableau* $\mathbf{T}$ *for* Φ *is a* **tableau proof** *of* Φ *iff it has no open branches.*

Theorem 1. ϕ *is an* S*-tautology iff there is a tableau proof for* $\{(N - S)\,\phi\}$*.*

This theorem is proved for classical logic, for example, in [12], for many-valued logics in [17].

2 Tableau Proofs with Constraints

In the light of the restrictions on signs and connectives placed in Section 1.2, we introduce, relatively to a set N of truth values, the following abbreviations for all $j \in N$ (cf. [14]):

$$\boxed{\geq j} := [j, 1] \cap N$$
$$\boxed{\leq j} := [0, j] \cap N$$

From now on until further notice let N be finite. Without loss of generality we assume that only signed formulas containing these signs are present on the initial tableau. Note that $([i_1, j_1] \cup \cdots \cup [i_m, j_m])\ \phi$ is satisfiable if and only if at least one of $[i_1, j_1]\,\phi, \ldots, [i_m, j_m]\,\phi$ is satisfiable. Moreover, $[i, j]\,\phi$ is satisfiable if and only if $\{\boxed{\geq i}\,\phi, \boxed{\leq j}\,\phi\}$ is satisfiable. Hence, in the finitely-valued case, we can reduce satisfiability of arbitrary signed formulas to not more than n satisfiability queries of the form $\{\boxed{\geq i}\,\phi, \boxed{\leq j}\,\phi\}$.

Consider a signed formula $\phi = \boxed{\geq i}\,F(\phi_1, \phi_2)$, where F is any 2-place connective. A signed formula of this type is satisfiable iff for some valuation v the value of $v(F(\phi_1, \phi_2))$ is greater than or equal than i. The key idea in the following is to leave i as well as the signs in the rule extensions uninstantiated. For example, we could write down a rule like

$$\frac{\boxed{\geq i}\,F(\phi_1, \phi_2)}{\begin{array}{c}\boxed{\geq i_1}\,F(\phi_1, \phi_2)\\ \boxed{\geq i_2}\,F(\phi_1, \phi_2)\end{array}}$$

For most instances of i, i_1, i_2, however, such a rule does not properly reflect the semantics of F, hence we must impose some additional constraints. Let us become a little bit more concrete and consider a signed formula $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$, where $\supset$ denotes n-valued Łukasiewicz implication, which can be defined as

$$i_1 \supset i_2 = \min\{1, 1 - i_1 + i_2\} \qquad \text{or, for } n = 3:$$

$\supset$	0	$\frac{1}{2}$	1
0	1	1	1
$\frac{1}{2}$	$\frac{1}{2}$	1	1
1	0	$\frac{1}{2}$	1

Let us denote with Ł the propositional logic that contains the implication connective as defined above as well as a negation which is defined by $\neg i = 1 - i$. Both definitions are valid for all sets of truth values. We indicate the cardinality of the set of truth values as a subscript, thus we have $Ł_3, \ldots, Ł_{\aleph_0}$ for three-valued, ..., infinitely-valued **Łukasiewicz logic**. An unsigned formula in Łukasiewicz logic is said to be satisfiable iff it is $\{1\}$-satisfiable.

From the semantic definitions we may derive a tableau rule for $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$. It has *provisos* which have to be satisfied in any proof the rule is used in. Provisos are used as well in classical quantifier rules, only that the proviso can be checked immediately in the case of quantifier rules, whereas we delay the check until tableau completion in the present case. Moreover, there may be different constraints associated with each extension. Let us call the rules of the new kind **constraint rules**. The rule for $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$ and its counterpart for $\boxed{\geq i}\,(\phi_1 \supset \phi_2)$ are given in Table 1. Note that the left extension of the rule for $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$ does not contain any formulas, but only the constraint '$i = 1$'.

Recall that the number of extensions for n-valued Łukasiewicz implication rules was up to n with the old rules, while now it is constant for arbitrary n.

Table 1. Constraint Rules for $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$ and $\boxed{\geq i}\,(\phi_1 \supset \phi_2)$.

$$\frac{\boxed{\leq i}\,(\phi_1 \supset \phi_2)}{i = 1 \quad \left|\begin{array}{ll} \boxed{\geq i_1}\,\phi_1 & i \leq \frac{n-2}{n-1} \\ \boxed{\leq i_2}\,\phi_2 & 1 - i_1 + i_2 = i \end{array}\right.} \qquad \frac{\boxed{\geq i}\,(\phi_1 \supset \phi_2)}{\begin{array}{ll} \boxed{\leq i_1}\,\phi_1 & \\ \boxed{\geq i_2}\,\phi_2 & 1 - i_1 + i_2 = i \end{array}}$$

Different values of n are handled in the constraints. Proof trees for the same formula and different N are identical modulo the constraints.

It is instructive to instantiate the premise of a rule and compute all solutions of its constraint system. Consider, for example, the rule for $\boxed{\geq \frac{1}{2}}\,(\phi_1 \supset \phi_2)$ in $Ł_5$. The constraint system consists of the single equation $1 - i_1 + i_2 = \frac{1}{2}$ which has to be solved over N. The pairs $(i_1, i_2) \in N^2 = \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}^2$ solving this equation are $\{(\frac{1}{2}, 0), (\frac{3}{4}, \frac{1}{4}), (1, \frac{1}{2})\}$.

Each different solution of the constraint system corresponds to a conventional rule extension, so we can backtranslate our example into a conventional rule with three extensions and no constraints:

$$\frac{\boxed{\geq \frac{1}{2}}\,(\phi_1 \supset \phi_2)}{\begin{array}{l|l|l} \boxed{\leq \frac{1}{2}}\,\phi_1 & \boxed{\leq \frac{3}{4}}\,\phi_1 & \boxed{\leq 1}\,\phi_1 \\ \boxed{\geq 0}\,\phi_2 & \boxed{\geq \frac{1}{4}}\,\phi_2 & \boxed{\geq \frac{1}{2}}\,\phi_2 \end{array}}$$

Eliminating the trivially satisfiable formulas $\boxed{\geq 0}\,\phi_2$ and $\boxed{\leq 1}\,\phi_1$ yields exactly the same rule for $\boxed{\geq \frac{1}{2}}\,(\phi_1 \supset \phi_2)$ as the method in [13, 17], namely the following:

$$\frac{\{\frac{1}{2}, \frac{3}{4}, 1\}\ (\phi_1 \supset \phi_2)}{\begin{array}{l|l|l} \{0, \frac{1}{4}, \frac{1}{2}\}\ \phi_1 & \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}\}\ \phi_1 & \\ & \{\frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}\ \phi_2 & \{\frac{1}{2}, \frac{3}{4}, 1\}\ \phi_2 \end{array}}$$

The constraint $1 - i_1 + i_2 = \frac{1}{2}$ is merely an implicit representation of the extensions in the conventional rule.

We must modify our earlier definitions slightly in order to accomodate linear constraints.

Definition 14 (Signed Formula). *Let $\phi \in L$, $i \in N \cup L_0$. Then we call the expressions $\boxed{\geq i}\,\phi$ and $\boxed{\leq i}\,\phi$* **signed formula**.

Definition 15 (Satisfiable Signed Formula). *A signed formula $\boxed{\geq i}\,\phi$ ($\boxed{\leq i}\,\phi$) is* **satisfiable** *iff $v(\phi) \geq v(i)$ ($v(\phi) \leq v(i)$) with the convention $v(i) = i$ for $i \in N$.*

We denote the set of **linear arithmetic constraints over** L_0 by C. Obviously, we may interpret a finite set of constraints $\Xi \subset C$ as the feasibility part of a bIP over N. Each solution of Ξ in turn can be interpreted as a valuation function. Hence, we may define satisfiability of mixed sets of signed formulas and constraints as follows:

Definition 16 (Satisfiable Sets of Formulas with Constraints). *Consider a mixed set of formulas and constraints $\Theta = \Phi \cup \Xi \subset L^* \cup C$ with $\Phi \subset L^*$, $\Xi \subset C$. We say Θ is* **satisfiable** *iff there is a valuation v that simultaneously satisfies all $\phi \in \Phi$ and solves Ξ.*

Let us denote the family of all finite sets of constraints by C^*. We change the definition of a tableau rule only slightly:

Definition 17 (Constraint Tableau Rule). *A* **constraint tableau rule** *is a finite sequence $\langle \phi, r, \Theta_1, \ldots, \Theta_r \rangle$, where $\phi \in L^*$, r is a positive integer, and for $1 \leq i \leq r$ $\Theta_i \in (((L^* \dot\cup (L^* \times L^*)) \times C^*) \dot\cup C^*)$.*

Note that we can leave the rest of the definition and the definition of a tableau itself unchanged.

Proposition 2. *For all valuations v and formulas $\phi_1, \phi_2 \in \text{Ł}_x$, where $x \in \{2, \ldots, \aleph_0\}$, v satisfies $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$ iff either $v(i) = 1$ or $v(i) < 1$, there is a v' that satisfies $\{\boxed{\geq i_1}\,\phi_1, \boxed{\leq i_2}\,\phi_2\}$ and differs from v at most in i_1, i_2, and $v(i) = 1 - v'(i_1) + v'(i_2)$ holds.*

Proof. If $v(i) = 1$ the proposition is immediate, so assume that $v(i) < 1$.

If v satisfies $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$, then $v(i) \geq v(\phi_1 \supset \phi_2) = 1 - v(\phi_1) + v(\phi_2)$. Hence, for suitable $j_1 \leq v(\phi_1), j_2 \geq v(\phi_2)$ we have that $v(i) = 1 - j_1 + j_2$. If we set $v'(i_1) := j_1$, $v'(i_2) := j_2$, and $v' = v$ otherwise, then all conditions are fulfilled.

Conversely, if $v(i) = 1 - v'(i_1) + v'(i_2)$ then, since v' satisfies $\{\boxed{\geq i_1}\,\phi_1, \boxed{\leq i_2}\,\phi_2\}$, $v(i) \geq 1 - v(\phi_1) + v(\phi_2) = v(\phi_1 \supset \phi_2)$. Therefore v satisfies $\boxed{\leq i}\,(\phi_1 \supset \phi_2)$.

From Proposition 2 we conclude immediately that the constraint tableau rule for $\boxed{\leq i}$ and Łukasiewicz implication (the rule on the left in Table 1) is sound and complete for any finite number of truth values, because then $i < 1$ iff $i \leq \frac{n-2}{n-1}$. For soundness and completeness of the rule on the right in Table 1, obviously, a similar proposition, which is similarly proved, is sufficient.

Remark 2. It is possible to weaken the constraint to $1 - i_1 + i_2 \leq i$ in the rule for $\boxed{\leq i}$ and to $1 - i_1 + i_2 \geq i$ in the rule for $\boxed{\geq i}$ and the resulting rules are still complete.

Before we define branch closure involving signs of the new kind we observe that with each branch of a completed tableau which is not yet tested for closure a system of linear inequations whose variables are ranging over N, in other words, a bIP problem over N is associated. Regarding the detection of closed branches

we note first that in a completed tableau it is sufficient to look for atomic closure. Also it is sufficient to look for *pairs* of contradictory formulas (instead of tuples) when signs are restricted as above. Moreover, closure can only occur between atomic formulas with a different type of sign, that is, it can never occur between formulas like $\boxed{\leq i_1}\,\phi$ and $\boxed{\leq i_2}\,\phi$. Thus, a branch can only be closed when two atomic formulas $\boxed{\leq i_1}\,\phi$, $\boxed{\geq i_2}\,\phi$ are present.[4] A branch is closed iff the signs have an empty intersection iff $i_1 < i_2$. We obtain thus for each tableau branch $\boldsymbol{B}$ a set of strict linear inequations

$$\{c_1 i_1 < d_1, \ldots, c_l i_l < d_l\},$$

such that solving *any one* of them results in the closure of $\boldsymbol{B}$, in other words, a disjunction of strict linear inequations. By definition, a branch is open when it cannot be closed. If we negate the disjunction

$$\bigvee_{k=1}^{l} (c_k i_k < d_k),$$

apply De Morgan's law and observe that $c_k i_k \not< d_k$ iff $c_k i_k \geq d_k$ we could define closure of $\boldsymbol{B}$ via the feasibility of the union of the bIP problems (A, B) and (C, D). There is, however, a more elegant way to deal with closure by adding the **atomic constraint rules** of Table 2 to each tableau system. The following proposition is obvious:

Proposition 3. *The atomic constraint rules are sound and complete.*

Table 2. Atomic constraint rules ($p \in L_0$).

$$\frac{\boxed{\geq i}\,p}{p \geq i} \qquad\qquad \frac{\boxed{\leq i}\,p}{i \geq p}$$

Definition 18 (Constraint Tableau, Closure). *A* **constraint tableau** *for Φ is built up using atomic constraint rules and sound and complete constraint rules. If $\boldsymbol{B}$ is a branch of a constraint tableau, then $\boldsymbol{B}$ is called* **closed** *iff the bIP (A, B) consisting of all the constraints on $\boldsymbol{B}$ is infeasible.*

This definition of closure has the advantage of being shorter than the first one which does not involve variables occurring *only* in the constraints, but it has the disadvantage of involving a potentially larger number of variables.[5] The definition of a tableau proof remains unchanged.

[4] Self-contradictory formulas cannot occur anymore, because the signs do no longer specify a concrete set of truth values.

[5] This phenomenon is characteristic for (M)IP representations, see [28, p. 8].

Theorem 2. *ϕ is an S-tautology iff there is a constraint tableau proof for $\{(N-S)\ \phi\}$.*

For the proof of this theorem we can use the usual machinery for completeness proofs of semantic tableau [12] with some modifications. Moreover, we assume that for each signed formula, there is a sound and complete constraint tableau rule with matching premise. Later we will see that this assumption is justified.

Definition 19 (Constraint Hintikka Set). *A constraint Hintikka set $\boldsymbol{H}$ is a set of signed formulas and constraints with the following properties:*

(H1) *The bIP defined by the set Ξ_H of all constraints occuring in $\boldsymbol{H}$ is feasible.*
(H2) *If $\boxed{\geq i}\, p{\in}\boldsymbol{H}$ ($\boxed{\leq i}\, p{\in}\boldsymbol{H}$), and $p \in L_0$, then $p \geq i \in\boldsymbol{H}$ ($i \geq p \in\boldsymbol{H}$).*
(H3) *If $\phi \in\boldsymbol{H}$, $\phi \in L^*$, ϕ not atomic, and R is a sound and complete tableau rule with a matching substitution $\sigma : L^* \to L^*$ such that $Prem(R)\sigma = \phi$, then at least one of the sets $Ext_1(R)\sigma, \ldots, Ext_{\alpha(R)}(R)\sigma$ is completely contained in $\boldsymbol{H}$.*

A similar definition of a constraint Hintikka set was given in a slightly more specialized context by Zabel [50, p. 180f].

Lemma 1. *Every constraint Hintikka set $\boldsymbol{H}$ is satisfiable.*

Proof. The constraint part Ξ_H of $\boldsymbol{H}$ is satisfiable by (H1). Assume v solves Ξ_H. Satisfiability of the formula part by v is proved via structural induction. Let $\boxed{\geq i}\, p{\in}\boldsymbol{H}$ be an atomic signed formula. By (H2), $p \geq i \in \Xi_H$, thus is satisfied by v. Then, by completeness of the atomic constraint rules, v also satisfies $\boxed{\geq i}\, p$. The other case is similar. Now, consider $\phi \in\boldsymbol{H}$, $\phi \in L^*$, ϕ not atomic. By our assumption on the existence of enough sound and complete rules and (H3), there is a sound and complete tableau rule R with a matching substitution $\sigma : L^* \to L^*$ such that at least one of the sets $\mathrm{Ext}_1(R)\sigma, \ldots, \mathrm{Ext}_{\alpha(R)}(R)\sigma$, say $\mathrm{Ext}_i(R)\sigma$, is completely contained in $\boldsymbol{H}$. By the induction hypothesis and satisfiability of Ξ_H, we know that v satisfies $\mathrm{Ext}_i(R)\sigma$. By completeness of R we have that v satisfies ϕ, and we are done.

Proof of Theorem 2: Assume ϕ is no S-tautology, therefore, $(N-S)\ \phi$ must be satisfiable by some valuation v. Consider any constraint tableau proof $\boldsymbol{T}$ for $\{(N-S)\ \phi\}$. As all rules used in $\boldsymbol{T}$ are sound, there must be at least one branch in $\boldsymbol{T}$ that is satisfied by v. In particular, v solves all constraints on this branch which, therefore, is open, and we have a contradiction.

Now assume there is no constraint tableau proof for $\{(N-S)\ \phi\}$. Hence, there is a completed constraint tableau $\boldsymbol{T}$ for $\{(N-S)\ \phi\}$ with at least one open branch. Take any open branch $\boldsymbol{B}$ of $\boldsymbol{T}$. By the definition of a completed tableau, $\boldsymbol{B}$ (considered as a the set of its labels) fulfills (H2) and (H3). But as $\boldsymbol{B}$ is also open ($H1$) is fulfilled as well. According to Lemma 1, $\boldsymbol{B}$ is satisfiable, and in particular ϕ is $(N-S)$-satisfiable, a contradiction to the fact that ϕ is an S-tautology.

We note that Theorem 2 obviously generalises to finite sets of signed formulas.

3 Example

Before we proceed, let us give an example of a tableau proof using constraint rules. We will give a proof of the formula $p \supset (q \supset p)$ in three-valued Łukasiewicz logic. In Figure 2 the conventional proof is shown, in Figure 3 the proof tree using constraint rules is displayed.

Note that the rule application to formula (3) in the tree in Figure 3 does yield only one branch, since adding the condition $i_2 = 1$ makes the constraint system on the branch unsolvable and thus closes it immediately.

The following feasibility part of a bIP problem corresponds to the only branch of the tree in Figure 3 (note that an equation is represented by two inequations; (iv) is obtained by adding (6) and (7)):

$$\begin{array}{lrrrrl}
\text{(ia)} & -i_1 & +i_2 & & & \geq -\frac{1}{2} \\
\text{(ib)} & +i_1 & -i_2 & & & \geq \frac{1}{2} \\
\text{(ii)} & & -i_2 & & & \geq -\frac{1}{2} \\
\text{(iiia)} & & -i_2 & -i_3 & +i_4 & \geq -1 \\
\text{(iiib)} & & +i_2 & +i_3 & -i_4 & \geq 1 \\
\text{(iv)} & -i_1 & & & +i_4 & \geq 0
\end{array}$$

To complete the proof we must show the infeasibility of this problem over $N = \{0, \frac{1}{2}, 1\}$. In the light of Remark 2 it suffices to consider only lines (ib), (ii), (iiib), and (iv).

This may seem not a great achievement if compared to constructing the conventional tableau proof, but we must take into account that, while the bIP problem does not become larger when n grows, the tableau becomes bigger and bigger (in the example it grows with $\mathcal{O}(n^2)$, but in general it grows with $\mathcal{O}(n^k)$, where k is the depth of the formula to be proved).

Another important point is that the system of inequations for each branch can be computed incrementally, while the tree is constructed, thus using information that was computed only once in more than one branch.

4 A Polynomial Reduction from Classical Logic Satisfiability to 0-1-IP

4.1 Linearization of Constraint Rules for Classical Logic

We will now see that constraint tableaux can in fact be linearized, in other words, a tableau can be translated into a single constraint system. In Operations Research merging of disjunctively connected IP problems has been extensively investigated. Given a set of IP problems over the same set of variables the task is to find a single IP problem, possibly involving auxiliary variables, such that it comprises exactly the disjunction of the solutions of the input systems. The discipline is called **disjunctive programming** and in the general case a

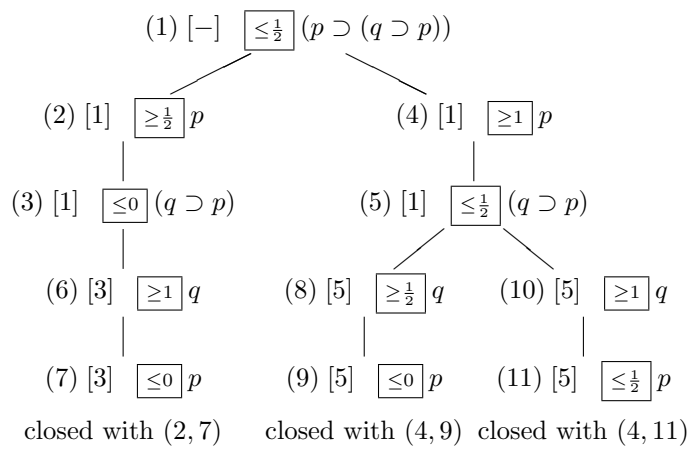

Fig. 2. A simple derivation in three-valued Łukasiewicz logic using standard tableau rules.

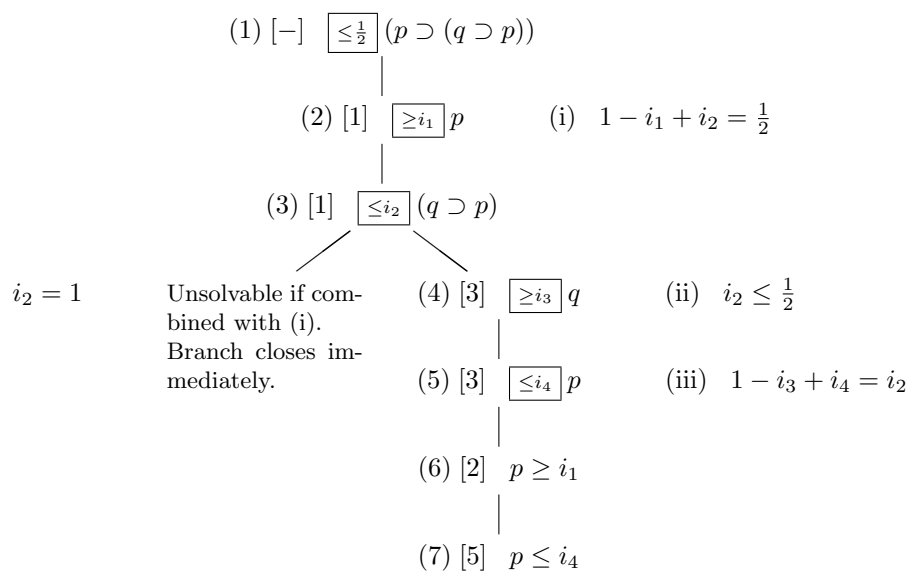

Fig. 3. A simple derivation in three-valued Łukasiewicz logic using constraint rules.

constraint system may involve linear inequations over mixed[6] variables, hence a bMIP problem.

Our first example will be a formulation of tableau rules for classical propositional logic which consists only of *linear rules*. It is given in Table 3. We use Smullyan's uniform notation [47] for classical connectives. Double negations are always eliminated before rule application. Note that only two rules introduce new sign variables at all and thus have associated constraints. The bIP problems resulting from tableaux constructed with these rules are in fact 0-1-IP problems.

Table 3. Classical tableau rules in disjunctive constraint formulation.

$\boxed{\leq i}\ \alpha$	$\boxed{\geq i}\ \alpha$	$\boxed{\geq i}\ \beta$	$\boxed{\leq i}\ \beta$
$\boxed{\leq i_1}\ \alpha_1 \quad i_1 + i_2 \leq i + 1$	$\boxed{\geq i}\ \alpha_1$	$\boxed{\geq i_1}\ \beta_1 \quad i_1 + i_2 \geq i$	$\boxed{\leq i}\ \beta_1$
$\boxed{\leq i_2}\ \alpha_2$	$\boxed{\geq i}\ \alpha_2$	$\boxed{\geq i_2}\ \beta_2$	$\boxed{\leq i}\ \beta_2$

4.2 Saving Variables in Constraint Rules

Several other representations of tableau rules than the one of Table 3 are possible and some of them pay in increased efficiency in an implementation. A variant of the two more complicated of the classical constraint rules where one sign variable is saved is shown in Table 4. To derive, for instance, the new β-rule from the old one, simply set i_1 to $i - i_2$ and i_2 to j. The prize is to admit linear expressions as signs, since we have to state the definition of the redundant variable, but this is just a technical point which does not cause any problems. The rules are complete, even when the constraints in brackets are omitted altogether. However, although redundant, the information can reduce time for solving the IP considerably. A similar optimization is possible for the rules of Łukasiewicz logic. We give them, including the rules for negated implications, in Table 5. The rules can be derived from the old rules in a similar way as in the classical case. Again, we can add some constraints which are not necessary for completeness, but could speed up computation. The similarities in the conclusions of the rules suggest that a similar schema as uniform notation in the classical case might be worked out for Łukasiewicz logic.

If we are to linearize the constraint rules of many-valued logics in general, and of Łukasiewicz logic in particular, then we must extend the formal framework of Section 2, because disjunctive programming requires the availability of variables that range over $\{0, 1\}$. We provide this generalisation in Section 5, which will also extend to infinitely-valued logics.

[6] Up to now we had only pure bIP problems where all variables ranged over a finite set, however, we will soon be in need of the more general case.

Table 4. Improved constraint rules for classical logic.

$$\frac{\boxed{\leq i}\ \alpha}{\begin{array}{c}\boxed{\leq i-j+1}\ \alpha_1\ (i \leq j)\\ \boxed{\leq j}\ \alpha_2\end{array}} \qquad \frac{\boxed{\geq i}\ \beta}{\begin{array}{c}\boxed{\geq i-j}\ \beta_1\ (i \geq j)\\ \boxed{\geq j}\ \beta_2\end{array}}$$

Table 5. Improved constraint rules for Łukasiewicz logic.

$$\frac{\boxed{\leq i}\ \phi_1 \supset \phi_2}{i=1 \left| \begin{array}{c}\boxed{\geq 1-i+j}\ \phi_1\\ \boxed{\leq j}\ \phi_2\end{array}\right.} \qquad \frac{\boxed{\geq i}\ \neg(\phi_1 \supset \phi_2)}{i=0 \left| \begin{array}{c}\boxed{\geq i+j}\ \phi_1\\ \boxed{\leq j}\ \phi_2\end{array}\right.} \qquad \frac{\boxed{\leq i}\ \neg(\phi_1 \supset \phi_2)}{\begin{array}{c}\boxed{\leq i+j}\ \phi_1\\ \boxed{\geq j}\ \phi_2\end{array}} \qquad \frac{\boxed{\geq i}\ \phi_1 \supset \phi_2}{\begin{array}{c}\boxed{\leq 1-i+j}\ \phi_1\\ \boxed{\geq j}\ \phi_2\end{array}}$$

4.3 Negation

In conventional tableaux there are two possible ways of dealing with negation and the same is true for constraint tableaux and many-valued logics. The first way was chosen in the rules above: negation operators in front of complex connectives are treated together with these connectives as α- or β-formulas, double negations are eliminated and negated atoms are transformed into inequations. This works only for some many-valued logics. The second method consists of viewing negations as proper connectives with their own rules and this works for all kinds of unary connectives. For instance, possible rules for Łukasiewicz negation are the following rules whose soundness and completeness is obvious:

$$\frac{\boxed{\leq i}\ \neg\phi}{\boxed{\geq 1-i}\ \phi} \qquad \frac{\boxed{\geq i}\ \neg\phi}{\boxed{\leq 1-i}\ \phi}$$

5 A Polynomial Reduction from Many-Valued Logic Satisfiability to bMIP

5.1 Infinitely-Valued Logics

In this section we demonstrate that constraint tableaux provide us with a decision procedure even for some infinitely-valued logics. The only approaches to theorem proving in infinitely-valued logics are, to our best knowledge, [3], [38, 39], and [50, pp. 175–189]. The first two are restricted to infinitely-valued Łukasiewicz

logic $Ł_{\aleph_0}$, the third is concerned with Post logics [43].[7] Our approach can handle *any* infinitely-valued logic, where the graph for all connectives is bMIP-representable, and queries are of the form $[c, d]$. Moreover, we claim that it can be implemented in an efficient way. To ease comparison with other approaches we use $Ł_{\aleph_0}$ as an example.

Let us first note that the left rule in Table 1 can be reformulated a little bit more redundant, but still complete:

$$\frac{\boxed{\leq i}\,(\phi_1 \supset \phi_2)}{i = 1 \quad \begin{array}{|l} \boxed{\geq i_1}\,\phi_1 \\ \boxed{\leq i_2}\,\phi_2 \; 1 - i_1 + i_2 = i \end{array}}$$

The only difference is the removal of $i \leq \frac{n-2}{n-1}$ in the right hand side constraint. A moment's thought reveals that this rule is still complete: when $i = 1$ some of the truth table entries in $\boxed{\leq 1}$ are now covered in both branches. This is not dramatic, however, since it happens also in the usual classical β-rules.

Next we note that this rule is sound and complete not only for finitely-valued Łukasiewicz logic, but also for $Ł_{\aleph_0}$, since N does not occur explicitly.

Applications of disjunctive programming methods [28] and subsequent simplification yields a linear formulation of the same rule:

$$\frac{\boxed{\leq i}\,(\phi_1 \supset \phi_2)}{\begin{array}{ll} \boxed{\geq i_1}\,\phi_1 & y \leq i, \quad i_1 \leq 1 - y \\ \boxed{\leq i_2}\,\phi_2 & 1 - i_1 + i_2 = y + i, \quad y \leq i_2 \end{array}}$$

where y is binary and i, i_1, i_2 range over N. If $y = 0$, then the constraint system simplifies to $\{0 \leq i, i_1 \leq 1, 0 \leq i_2, 1 - i_1 + i_2 = i\}$. Since the first three constraints are redundant, this corresponds to the constraint system in the right extension of the rule above. If $y = 1$, then the constraint system simplifies to $\{i = 1, i_1 = 0, i_2 = 1\}$ which causes the conclusion to become trivially satisfiable,

[7] Zabel's [50, pp. 175–189] work is closely related to our own in the following sense: he starts out from signed semantic tableaux as well, and then he enriches the tableau system with an arithmetic constraint language. He even uses signs of the form $\boxed{\geq i}$ and $\boxed{\leq i}$, altough in a different notation. There are, however, a number of crucial differences: (i) Zabel uses $\mathbb{N} \cup \{\omega\}$ as the domain of truth values and $\mathbb{N}$ as the domain of the constraint variables, whereas we use $\mathbb{Q} \cap [0, 1]$ for both; (ii) Zabel does not employ disjunctive programming techniques, hence he does not achieve a reduction from infinitely-valued logic satisfiability to 0-1-MIP as we do in the present section; (iii) Zabel does not say how to obtain constraint tableau systems for different logics than Post logic, whereas we will subsequently point out how to obtain them for any bMIP-representable logic in a systematic way. Finally, we remark that the crucial ideas of the present section were already published in [15], and that Zabel's work was carried out independently.

and therefore, it is identical with the left extension of the rule above. Hence, y selects, depending on its value, either the left or the right rule extension. For this reason y is called **control variable**. The rule on the right in Table 1 as well as rules for negation stay unchanged.

To prove validity of a formula ϕ in $Ł_{\aleph_0}$ we construct a constraint tableau with root $\boxed{\leq c}\,\phi$. Since all rules are linear there is only a single branch from which we extract a set of mixed integer/linear inequalities as before. Now we minimize c with respect to these. If this 0-1-MIP problem gives $c = 1$ as a solution, we know that $\boxed{\leq c}\,\phi$ is unsatisfiable for $c < 1$, hence $v(\phi) = 1$ for all valuations v.

In this manner we can translate every many-valued satisfiability problem from $Ł_{\aleph_0}$ into a single 0-1-MIP problem whose integer part has not more variables than the input formula has connectives. Let us formalise the aforegoing considerations.

Let B_0 be a countable set of variables called **control variables** that is disjoint with L_0. Let C^+ be the the set of **linear arithmetic constraints over** $L_0 \cup B_0$. If we interpret members of B_0 as variables ranging over $\{0, 1\}$, and $N = \mathbb{Q} \cap [0, 1]$, then we may regard a finite set of constraints $\Xi^+ \subset C^+$ as the feasibility part of a 0-1-MIP.

We extend valuations v to $L_0 \cup B_0$ such that $v(B_0) \subseteq \{0, 1\}$, and hence define satisfiability of constraints over $L_0 \cup B_0$, tableau rules involving constraints over $L_0 \cup B_0$, and soundness and completeness of those, exactly as before (cf. Defs. 16, 17, and 9).[8]

Additionally, we may extend constraint Hintikka sets $\boldsymbol{H}$ (cf. Def. 19) to constraint sets $\Xi_H^+ \subset C^+$. It is easy to see that Lemma 1 still holds in this more general context.

The definition of a tableau can stay unchanged, but the following terminology will be useful:

Definition 20 (MIP problem associated with a formula). *Consider an infinitely-valued bMIP-representable logic $\mathcal{L}$, and assume that we have a sound and complete set of* linear *constraint tableau rules for it. Let, furthermore, $\boldsymbol{T}$ be a completed tableau for $\boxed{\leq c}\,\phi \in L^*$. As all rules are linear, $\boldsymbol{T}$ has at most one branch. Moreover, in the case $c = 1$ the signed formula $\boxed{\leq c}\,\phi$ is always satisfiable, thus by soundness of the tableau rules the set Ξ_T^+ of all constraints occurring in $\boldsymbol{T}$ must be feasible, and hence $\boldsymbol{T}$ has exactly one branch. Let us call the 0-1-MIP that consists of Ξ_T^+ with objective function c an* **MIP problem associated with** *ϕ, and the minimal c' that solves it its solution; call the 0-1-MIP that consists of Ξ_T^+ with objective function $-c$ an* **MIP problem co-associated with** *ϕ, and the maximal c' that solves it its solution.*

Theorem 3. *Let $\mathcal{L}$ be an infinitely-valued bMIP-representable logic, $\phi \in L^*$, $c_0, d_0 \in N$.*

[8] According to our previous considerations, the linear rule for $\boxed{\leq i}$ and Łukasiewicz implication given above, as well as the rule for $\boxed{\geq i}$ and Łukasiewicz implication from Table 1, and the negation rules are sound and complete in this sense.

1. *ϕ is a $[c_0, 1]$-tautology iff $c_0 \leq c'$ for the solution c' of an MIP problem associated with ϕ.*
2. *ϕ is a $[0, d_0]$-tautology iff $d_0 \geq d'$ for the solution d' of an MIP problem co-associated with ϕ.*

Proof. We prove only the first statement, the proof of the second is similar. Let ϕ be a $[c_0, 1]$-tautology. Let c' be the solution of an MIP problem associated with ϕ. Recall from the previous definition that this solution exists; moreover, since the objective function is one-dimensional, it must be unique. Assume that $c' < c_0$. Now consider the tableau $\boldsymbol{T}$ for $\boxed{\leq c}\,\phi$. Let $\boldsymbol{T}^{c'}$ be tableau obtained by replacing every occurrence of c in $\boldsymbol{T}$ with c'. Since c' was a solution to the MIP problem associated with $\boldsymbol{T}$, the constraint set $\Xi^+_{T^{c'}}$ that stems from $\boldsymbol{T}^{c'}$ must be feasible. Hence, (the single branch of) $\boldsymbol{T}^{c'}$ constitutes a constraint Hintikka set. Then, by Lemma 1, $\boxed{\leq c'}\,\phi$ must be satisfiable, thus there is a valuation v such that $v(\phi) \in [0, c']$. This contradicts the fact that ϕ is a $[c_0, 1]$-tautology.

Now let us turn to the other direction, that is, assume that c' with $c_0 \leq c'$ is the solution of an MIP problem associated with ϕ, but ϕ is no $[c_0, 1]$-tautology. From the latter we deduce that $\boxed{\leq c''}\,\phi$ must be satisfiable with a suitable $c'' < c_0$. By the soundness (and linearity) of the tableau rules we have that the constraint system $\Xi^+_{T^{c''}}$ associated with the tableau $\boldsymbol{T}^{c''}$ must be solvable, so c'' must be a solution of Ξ^+_T, and hence no number greater or equal than c_0 can possibly constitute a minimal solution. This contradicts the assumption.

Remark 3. The previous theorem tells us how to ensure that a formula is a $[0, d_0]$- or a $[c_0, 1]$-tautology. More complicated queries involve the solution of more than one MIP problem, because a two-dimensional objective function such as $f(c, d) = c - d$ gives no longer unique solutions for c and d.

In order to determine, for instance, that ϕ is a $[c_0, d_0]$-tautology, we need to establish that the solution c' of the MIP problem associated with ϕ is greater or equal than c_0, and that the solution d' of the MIP problem co-associated with ϕ is smaller or equal than d_0. Then ϕ must be a $[c', d']$-tautology which implies its being a $[c_0, d_0]$-tautology.

Proposition 4. *There is a set of linear, sound, and complete constraint tableau rules for infinitely-valued Łukasiewicz logic such that the size of an associated MIP problem for every $\phi \in Ł_{\aleph_0}$ is $\mathcal{O}(|\phi|)$. In particular, the number of 0-1-variables occurring in the MIP is not greater than $\frac{1}{2}|\phi|$.*

Proof. Immediate from the rule set given above, if we note that subformulas are copied in none of the rules. 0-1-variables are only introduced one at a time in the $\boxed{\leq i}$ rule for implication, and thus are bounded by the number of implication connectives.

By combining the previous proposition with Proposition 1 and Theorem 3 for the case $Ł_{\aleph_0}$ and $c_0 = 1$ we obtain

Corollary 1. $SAT_{Ł_{\aleph_0}} \in NP$.

This result was obtained in [36] in a different way using a characterization of Łukasiewicz functions given by McNaughton [35]. Our completely different method in our opinion is simpler and more intuitive, at least from the point of view of computer science and AI, since it draws on standard techniques of linear optimization and automated theorem proving. Moreover, it renders itself also to other logics for which such results as the one of McNaughton do not exist.[9]

As before, the rules for Łukasiewicz logic can be simplified (the bracketed constraint is not necessary for completeness):

$$\frac{\boxed{\leq i}\ \phi_1 \supset \phi_2}{\begin{array}{c}\boxed{\geq 1-i+j-y}\ \phi_1 \\ \boxed{\leq j+y}\ \phi_2\end{array}} \quad \begin{array}{c} y \leq i \\ (j \leq i)\end{array} \qquad \frac{\boxed{\geq i}\ \phi_1 \supset \phi_2}{\begin{array}{c}\boxed{\leq 1-i+j}\ \phi_1 \\ \boxed{\geq j}\ \phi_2\end{array}}$$

We close this section with an example. We prove that the formula from the example in Section 2 is a tautology even in $Ł_{\aleph_0}$. The tableau is

$$\begin{array}{ll}
\boxed{\leq c}\ p \supset (q \supset p) & \\
\quad | & \\
(1)\ \boxed{\geq 1-c+j-y}\ p & y \leq c\ \ (< 1 \rightsquigarrow y = 0) \\
\boxed{\leq j+y}\ q \supset p & (2)\ j \leq c \\
\quad | & \\
(3)\ \boxed{\geq 1-j+k-z}\ q & z \leq j\ \ (< 1 \rightsquigarrow z = 0) \\
(5)\ \boxed{\leq k+z}\ p & (4)\ k \leq j
\end{array}$$

With $y \leq c$ and $c < 1$ we can fix $y = 0$, since y is a binary variable, which simplifies the resulting MIP. A similar argument goes through for z. The numbered inequations become part of the characteristic MIP for the problem

$$\begin{array}{lrrrrl}
(1) & & p & -j & +c & \geq 1 \\
(2) & & & -j & +c & \geq 0 \\
(3) & q & & +j & -k & \geq 1 \\
(4) & & & j & -k & \geq 0 \\
(5) & & -p & & +k & \geq 0
\end{array}$$

[9] The other direction, NP-hardness, was shown, too, in [36]. The idea is to define for each set of propositional variables $p_1, \ldots p_k$ an $Ł_{\aleph_0}$-formula $\texttt{two}(p_1, \ldots p_k)$ such that $\texttt{two}(p_1, \ldots p_k)$ is satisfiable in $Ł_{\aleph_0}$ iff $p_1, \ldots p_k$ are assigned binary truth values. Then for any formula ϕ which contains the propositional variables $p_1, \ldots p_k$ it is true that ϕ is satisfiable in classical logic iff $\texttt{two}(p_1, \ldots p_k) \supset \phi$ is satisfiable in $Ł_{\aleph_0}$. If $\texttt{two}$ has polynomial size in k this property reduces SAT to $\mathrm{SAT}_{Ł_{\aleph_0}}$. In $Ł_{\aleph_0}$ the function $\texttt{two}$ is a bit awkward to define, since there is no connective corresponding to truth value set complement. The technique works also for other many-valued logics.

which is in fact only an LP problem and easily seen to be infeasible if $c < 1$[10] and feasible for $c = 1$. Simplifications due to obvious variable fixings as they were possible above, occur quite often in practice and count to the strengths of the MIP approach.

5.2 A More General View

Theorem 3 provided us with a general soundness and completeness result for infinitely-valued logics provided that a set of sound, complete, and linear constraint tableau rules is available for all connectives and signs of the form $\boxed{\geq i}$, $\boxed{\leq i}$.

In this section we (i) say why this assumption is justified for all logics such that the graph of every connective f is bMIP-representable (recall that this was the overall assumption), (ii) we point out which logics typically fulfill this condition, and (iii) we lay down what kind of special results have to be proved for each logic.

We begin with the observation that in presence of signs $\boxed{\geq i}$ and $\boxed{\leq i}$ we need to be able to represent the sets epigraph$(f) = \{(i_1, \ldots, i_k, j) | f(i_1, \ldots, i_k) \leq j\}$ and hypograph$(f) = \{(i_1, \ldots, i_k, j) | f(i_1, \ldots, i_k) \geq j\}$ as bMIPs. For if we have such representations, then we can easily derive a sound, complete, and linear tableau rule from it.

If we want to compute, for instance, a rule for $\boxed{\geq i}\, f(i_1, i_2)$, then we take a bMIP that represents hypograph(f), collect all lines in which i_1 and i_2 occur, say $I_1^1 \geq i_1, \ldots, I_{k_1}^1 \geq i_1, I_{k_1+1}^1 \leq i_1, \ldots, I_{l_1}^1 \leq i_1, I_1^2 \geq i_2, \ldots, I_{k_2}^2 \geq i_2, I_{k_2+1}^2 \leq i_2, \ldots, I_{l_2}^2 \leq i_2$, and, consequently, we take $\boxed{\leq I_1^1}\, i_1, \ldots, \boxed{\leq I_{k_1}^1}\, i_1, \boxed{\geq I_{k_1+1}^1}\, i_1, \ldots, \boxed{\geq I_{l_1}^1}\, i_1, \boxed{\leq I_1^2}\, i_2, \ldots, \boxed{\leq I_{k_2}^2}\, i_2, \boxed{\geq I_{k_2+1}^2}\, i_2, \ldots, \boxed{\geq I_{l_2}^2}\, i_2$ as the conclusion of the rule with the rest of the bMIP as the constraint part of the rule. Similarly for $\boxed{\leq i}\, f(i_1, i_2)$. If i_1 and i_2 occur in the same line, say $i_1 \geq I + i_2$, then we introduce a new rational variable j, and replace the above line by the two lines $i_1 \geq j$ and $j \geq I + i_2$. This operation obviously preserves satisfiability. Hence, we can always guarantee that each line of the bMIP contains at most one occurrence of either i_1 or i_2.

The existence of bMIP-representations of epigraph(f) and hypograph(f) is equivalent to the existence of a bMIP-representation of graph(f) [28], hence the restriction was introduced for the graph.

---

[10] Obtain $-p + j \geq 0$ by adding (4) and (5); from this and (1) we obtain $c \geq 1$ which contradicts $c < 1$. Note that (3) is not needed in this derivation. In fact it is probably possibly to prove an analogon to the pure literal rule known from resolution. On the other hand, we made use of (4) upon which we previously remarked that it is not needed for completeness. A derivation not using (4) is as follows: from (3) and the boundary condition $-p \geq -1$ infer (4), then reason as above. One says that (3) *reduces to* (4) [25]. Obviously, (3) is now indispensible, hence a pure literal rule and minimisation of the number of constraints exclude each other.

Now we turn to the question of giving some typical examples. Among logics that *cannot* be represented by a bMIP, are all logics with discontinous connectives. Take, for example, the connective defined as

$$d(i) := \begin{cases} 0 \; i < 0.5 \\ 1 \text{ otherwise} \end{cases}$$

whose graph is not bMIP-representable. On the other hand, the hypograph of d *is* bMIP-representable (one possibility is $\{\boxed{\geq \frac{1}{2}y}\, j, i \leq y\}$, where y is binary), so if in the bMIP-representations of all premises only the same kind of signs do occur, the restriction on connectives f can be relaxed to bMIP-representability of either the epigraph or the hypograph.

Another class of connectives that are not bMIP-representable are those whose definition involves non-linear expressions, for example $e(i) = i^2$.

Both kinds of connectives occur frequently in mathematical modelling, but we want to emphasize that they do not necessarily limit our approach. Such connectives limit the scope of linear optimization methods in general which we have chosen as a basis for our reduction as they are well understood and can be implemented in a fairly efficient way. Without pointing out the details we remark that a broader class of connectives could be handled on the basis of non-linear optimization methods. As the latter are usually very restricted with respect to performance we went not further along this possible generalization.

As to the positive examples, we have already seen that Łukasiewicz connectives are bMIP-representable; the same holds for those of Post logic [43] and of the fuzzy logic system of Lee & Chang [32]. In general, connectives whose graph can be represented as the union of a finite number of convex polyhedra are bMIP-representable [28] (note that this is not true for d above, since the part where $d \equiv 0$ is not a polyhedron).

Finally, let us summarize which kinds of special results must be proved for each logic in order for having a sound and complete translation into bMIP.

As already mentioned we need bMIP-representations of epigraph(f) and hypograph(f) from which we derive rules. For all of these rules analoga of Proposition 2 which are sufficient for the soundness and completeness of the rules must be proved. If, moreover, we want to show that a logic is contained in NP, we must ensure that no rule "duplicates" the subformulas of the formula in the premise. Assume we have a bMIP-representation of $\boxed{\geq i}\, f(i_1, i_2)$, wherein without loss of generality i_1 occurs in more than one line. If all occurrences of i_1 are positive, that is, they are of the form $i_1 \geq I$, then we can apply a similar trick as above: substitute all occurrences of i_1 with a new (rational) variable j, then add the constraint $i_1 \geq j$; similarly, if all occurrences of i_1 are negative. This does not alter the solutions, but the i_1 occurs only once now, and hence i_1 occurs only once in the conclusion of the corresponding constraint tableau rule.

If, however, i_1 occurs both negatively and positively in every possible bMIP-representation, then we cannot in general avoid having two copies of i_1 (with complementary signs) in the conclusion of the corresponding constraint tableau rule. This means that, in general, the bMIP associated with a formula ϕ is not

polynomially bounded in the length of ϕ. For logics, where this is the case, our reduction does not constitute an NP containment proof. It is an open question, whether there are bMIP-representable logics (in our sense) whose satisfiability problem actually is not in NP.

As an example for the above considerations we may take the classical equivalence operator $\leftrightarrow$. One possible constraint tableau rule is:

$$\frac{\boxed{\geq i}\ \phi_1 \leftrightarrow \phi_2}{\begin{array}{c} \boxed{\geq i-j}\ \phi_1 \\ \boxed{\geq i-j}\ \phi_2 \\ \boxed{\leq 1+i-2j}\ \phi_1 \\ \boxed{\leq 1+i-2j}\ \phi_2 \end{array}} \qquad j \leq i$$

It is impossible to obtain a rule, where ϕ_1 and ϕ_2 are not copied in the conclusion. The classical equivalence connective can easily be generalized to infinitely-valued logics (for instance, via $i_1 \leftrightarrow i_2 := \min\{\max\{1-i_1, i_2\}, \max\{i_1, 1-i_2\}\}$). It is known that the satisfiability problem of classical logic even with equivalences is in NP. This is due to the fact that classical equivalence is associative, hence we can always assume that a chain of equivalences is made up of equal pieces. In this case the decomposition of an equivalence chain involves a logarithmic number of rule applications, and therefore, even with the duplications in the rule above, the whole translation is polynomial. It easy to conceive an infinitely-valued connective which is slightly different than equivalence, and no longer associative. It would be interesting to investigate the complexity of the satisfiability problem of a logic that contains such a connective.

Another question that is related to the complexity of the satisfiability problem of logics is how many new 0-1 variables must be introduced at least in the bMIP-representation of a connective. This number grows with the number of convex polyhedrons that are needed to characterize the graph of a connective.

From this point of view the complexity of classical logic, Post logic, Łukasiewicz logic and strong Kleene logic is the same, but it is obvious that there are many-valued logics with an arbitrarily high complexity in this sense. This complexity measure reflects *geometrical* properties of the truth tables. All we can say of the mentioned logics in terms of traditional complexity classes is that they are NP-complete, because the numbers of 0-1-variables in their bMIP characterizations are polynomially related.

5.3 Finitely-Valued Logics

In the finitely-valued case we have two options: first, Definition 20 and Theorem 3 are applicable to finitely-valued logics and bIPs as well. On the other hand, in the finitely-valued case we do not need to minimize an objective function, because of the following obvious fact:

Proposition 5. *The bIP* (B, h) *with objective function* $f(c) = c$ $(f(d) = -d)$ *associated (co-associated) with* ϕ *has a solution* $c' \geq c_0 > 0$ $(d' \leq d_0 < 1)$ *iff* $(B, h) \cup \{c = c_0 - \frac{1}{n-1}\}$ $((B, h) \cup \{d = d_0 + \frac{1}{n-1}\})$ *is infeasible.*

Therefore, in order to establish that ϕ is a $[c_0, 1]$-tautology, in the finitely-valued case it is sufficient that the bIP $(B, h) \cup \{c = c_0 - \frac{1}{n-1}\}$ as defined above is infeasible, and similarly for $[0, d_0]$-tautologies.

It is easy then to disjunctively combine these bIPs such as to reflect queries mentioned in Section 1.2 on page 4 which, in the finitely-valued case, means arbitrary queries.

6 Some Connections to CNF Transformation Algorithms

In this section we are only concerned with two-valued propositional logic.

It turns out that for classical logic, there is a close relationship between our IP translation and certain algorithms for translating formulas in CNF.

First let us note that if ϕ is already in CNF, then our translation using the rules from Table 4 coincides with the standard translation of CNF formulas into IP. In this standard translation (cf. [24]) each clause of the form

$$p_1 \vee \cdots \vee p_k \vee \neg p_{k+1} \vee \cdots \vee \neg p_m$$

is represented by a linear inequation

$$p_1 + \cdots + p_k + (1 - p_{k+1}) + \cdots + (1 - p_m) \geq 1. \tag{1}$$

In our notation, in order to translate a clause, we must expand the constraint tableau with root

$$\boxed{\geq 1}\, p_1 \vee (\cdots \vee (p_k \vee (\neg p_{k+1} \vee (\cdots \vee \neg p_m) \cdots)))$$

according to the β-rule in Table 4. This process yields after $m - 1$ β-rule applications, $m - k$ negation rule applications, and m atomic constraint rule applications the following IP:

$$\begin{aligned} p_1 &\geq 1 - j_1 \\ p_2 &\geq j_1 - j_2 \\ &\cdots \\ 1 - p_{m-1} &\geq j_{m-2} - j_{m-1} \\ 1 - p_m &\geq j_{m-1} \end{aligned}$$

Now, simple addition of all lines gives at once (1). But what if the input is *not* in CNF? Consider an algorithm for translating an arbitrary formula into a set of linear equations which is given by the following set of rules:

$$\frac{D;\beta \geq 1}{\begin{array}{c} D+j;\beta_1 \geq 1 \\ (1-j);\beta_2 \geq 1 \end{array}} \qquad \frac{D;\alpha \geq 1}{\begin{array}{c} D;\alpha_1 \geq 1 \\ D;\alpha_2 \geq 1 \end{array}} \qquad \frac{D;\neg\neg\phi \geq 1}{D;\phi \geq 1}$$

$$\frac{D;p \geq 1}{D+p \geq 1} \qquad \frac{D;\neg p \geq 1}{D+(1-p) \geq 1}$$

The notation is easily explained: D is a linear expression generated so far, j is a new variable and we start with '$0;\phi \geq 1$' to convert a formula ϕ into CNF. Then the rules are recursively applied until all connectives are eliminated. The rules are directly derived from the rules in Tables 3 and 4 and are, therefore, sound and complete. If D was of the form (1), then, after a suitable rearrangement of subterms, the linear expression in the conclusion of each rule is of the same form, hence the resulting IP corresponds to a formula in CNF.

We can learn more about the nature of that formula if we write down the very same rules in logic syntax instead of IP syntax:

$$\frac{D \vee \beta}{\begin{array}{c} D \vee q \vee \beta_1 \\ \neg q \vee \beta_2 \end{array}} \qquad \frac{D \vee \alpha}{\begin{array}{c} D \vee \alpha_1 \\ D \vee \alpha_2 \end{array}} \qquad \frac{D \vee \neg\neg\phi}{D \vee \phi} \tag{2}$$

Here, D is simply a clause, q a new variable and to convert ϕ we start with ϕ where $D = \Box$. The process terminates when a literal is reached. These rules constitute an algorithm for translating arbitrary formulas into CNF and which has the additional property that it needs only linear time and space. The existence of such algorithms, sometimes called *structure preserving clause form translations*, is well known and due to Tseitin [48]. They have been investigated, for instance, in [7].

It is possible to design linear time CNF translation algorithms in the style of [7] for arbitrary finitely-valued logics using *inverted tableau rules*, see [18]. These translations produce clauses over *signed literals* and there is a signed version of resolution that operates on them. It is not clear at this point, what the nature of the connection between signed clauses and the translation into IP problems stemming from the translation in the present paper is.

Finally, we want to point out the possibility of finding new normal form results for many-valued logics with the techniques presented here. In the left part of Figure 4 the connections between formulas in CNF, CNF transformation algorithms and our translation of formulas into 0-1 IP problems for two-valued logic is summarized. In the right part the picture is generalized to infinitely-valued logic, however, in the many-valued case our IP translation in general does not result in an IP problem corresponding to clauses. The question is, whether there is a class of many-valued formulas in some normal form that corresponds to the MIP problems.

For Łukasiewicz logic, a kind of normal form has been recently established [38, 39]. There, a Łukasiewicz function is decomposed into a sum of so-called

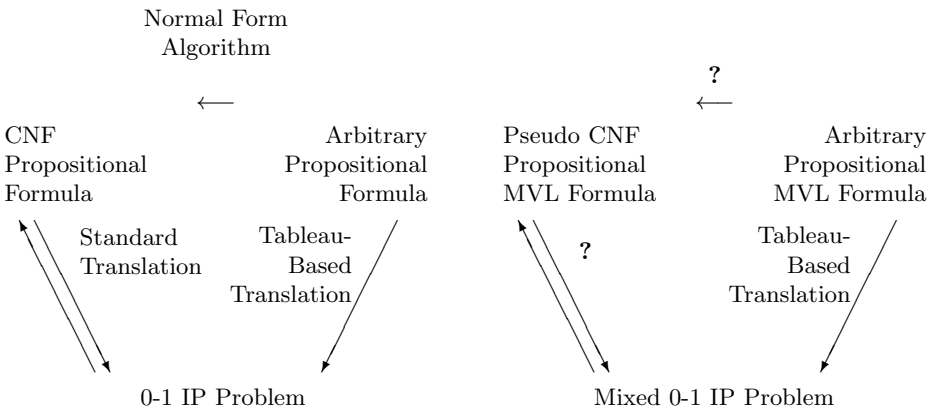

Fig. 4. Connection between CNF and IP.

Schauder hats which themselves are sup-inf expressions over certain linear functions. In [34] it was shown that there is no direct relationship between this normal form and the MIP translation presented here. While [38, 39] sheds light on the geometrical structure of Łukasiewicz functions and indeed establishes a kind of normal form for them, the reduction to MIP established in the present work does not exhibit any structure of Łukasiewicz functions and indeed is equally well applicable to quite different logics. In fact our method could be used as a *preprocessing* step for [39]. The latter then could be seen as a method of bringing structure into the MIPs that correspond to Łukasiewicz functions. For instance, the starring operations on triangulations that are used in [39] can be interpreted as certain transformations on MIPs.

7 Conclusion & Further Research

The main contribution of this paper is the generalization of the relationship between two-valued logic and 0-1-IP to many-valued logic and bMIP. Two immediate consequences are (i) the possibility to build relatively efficient decision procedures for finitely and infinitely-valued logics (in Table 6 we summarized a few results for $Ł_{\aleph_0}$ achieved with an MIP-based[11] satisfiability checker) and (ii) schematic NP containment proofs for infinitely-valued logics. We mentioned a novel way of looking at the complexity of many-valued logics and it might be possible to use the presented translation technique for deriving new normal form results.

Directions of future research include the following:

[11] It also is easy to implement constraint tableaux with constraint logic programming technology. Even though the performance in general cannot compete with MIP code this offers considerable flexibility, see [19].

Table 6. Computational results for an MIP based satisfiability checker for $Ł_{\aleph_0}$. Ax1 is the example in Section 5.1. Λ_i are the sums of Schauder formulas based on Farey partitions taken from [40]. The formula Λ_2 consists of more than 60 binary connectives. The results were obtained with our own MIP implementation written in C^{++} on a Sun Sparc 10/20. They include the time needed for translation.

Problem Name	Input Length	Resulting MIP 0-1 Vars.	Resulting MIP Rat. Vars.	Time (secs)
$Ax1$	5	2	5	0.060
Λ_0	4	1	3	0.030
Λ_1	24	5	7	0.130
Λ_2	181	27	37	13.830

- While going from search trees to MIP representations structural information is lost. We might propagate such information from the tableau to the MIP representation, for instance, by specifying a partial order on variables which is then used to determine the order in which they are fixed when solving the MIP by a branch-and-bound method.
- The preceding treatment of finitely-valued logics is centered around semantic tableau and translates logical satisfiability into solving of bIPs. In order to obtain a faster method, one should consider to enhance a Davis-Putnam-like procedure with linear constraints in order to make it work for finitely-valued logics.
- IP is hard to do while LP is not. It has been shown in other contexts that with certain inference rules (for example unit resolution) it is safe to substitute LP for IP. It would be interesting to identify such situations in the present context.
- The technique may well be applicable to other non-classical logics, such as temporal logics. First results are reported in [20].
- Using IP solvers as ground satisfiability checkers one could design a first-order prover that can accomodate successful strategies from three different theorem proving paradigms (semantic tableaux; resolution; IP) in a homogenous implementation framework. Ideas from systems such as CLIN [41, 33] could be further developed within such a systems. Investigations and a first implementation in this direction are under way (a preliminary report may be found in [44]).
- Apply fast satisfiablity checkers for many-valued logics to verification of integrated circuits on the switch level [9, 21].

Acknowledgements

I am grateful to Daniele Mundici and Klaus Ries for valuable discussions and suggestions. I appreciated the precise remarks of an anonymous referee which led to some improvements.

References

1. H. Barringer, J.H. Cheng, and C.B. Jones. A logic covering undefinedness in program proofs. *Acta Informatica*, 21:251–269, 1984.
2. Peter Barth and Alexander Bockmayr. Solving 0–1 problems in CLP($\mathcal{PB}$). In *Proc. CAIA, Orlando, Florida*, pages 263–269. IEEE Press, 1993.
3. Mervin G. Beavers. Automated theorem proving for Łukasiewicz logics. Manuscript of talk given at 1991 Meeting of Society for Exact Philosophy, Victoria, Canada, May 1991.
4. C. Bell, A. Nerode, R. Ng, and V.S. Subrahmanian. Implementing deductive databases by mixed integer programming. In *Proc. ACM Symposium on Principles of Database Systems*, pages 283–292, 1992.
5. C. Bell, A. Nerode, R. Ng, and V.S. Subrahmanian. Mixed integer programming methods for computing nonmonotonic deductive databases. *JACM, to appear*, 1994.
6. Dines Bjørner, Manfred Broy, and Igor V. Pottosin, editors. *Formal Methods in Programming and Their Applications*, volume 735 of *Lecture Notes in Computer Science*. Springer Verlag, 1993.
7. Thierry Boy de la Tour. Minimizing the number of clauses by renaming. In Mark E. Stickel, editor, *Proc. 10th International Conference on Automated Deduction, Kaiserslautern*, pages 558–572. Springer, LNCS 449, July 1990.
8. Manfred Broy, Christian Facchi, Radu Grosu, Rudi Hettler, Heinrich Hussmann, Dieter Nazareth, Franz Regensburger, and Ketil Stølen. The requirement and design specification language Spectrum, an informal introduction, version 1.0. Technical report, Institut für Informatik, Technische Universität München, March 1993.
9. Wolfram Büttner, Klaus Estenfeld, Richard Schmid, Hans-Albert Schneider, and Erik Tidén. Symbolic constraint handling through unification in finite algebras. *Applicable Algebra in Engineering, Communication and Computing*, 1:97–118, 1990.
10. Janusz Czelakowski. Algebraizability of logic and the deduction theorem. Lecture Notes Fourth Summer School on Logic, Language and Information, Colchester/England, August 1992.
11. George B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, 1963.
12. Melvin C. Fitting. *First-Order Logic and Automated Theorem Proving*. Springer, New York, 1990.
13. Reiner Hähnle. Towards an efficient tableau proof procedure for multiple-valued logics. In *Proceedings Workshop on Computer Science Logic, Heidelberg*, pages 248–260. Springer, LNCS 533, 1990.
14. Reiner Hähnle. Uniform notation of tableaux rules for multiple-valued logics. In *Proc. International Symposium on Multiple-Valued Logic, Victoria*, pages 238–245. IEEE Press, 1991.
15. Reiner Hähnle. A new translation from deduction into integer programming. In Jacques Calmet and John A. Campbell, editors, *Proc. Int. Conf. on Artificial Intelligence and Symbolic Mathematical Computing AISMC-1, Karlsruhe, Germany*, pages 262–275. Springer, LNCS 737, 1992.
16. Reiner Hähnle. Automated deduction and integer programming. In *Yearbook of the Kurt-Gödel-Society*. Kurt-Gödel-Society, Vienna/Austria, 1993.
17. Reiner Hähnle. *Automated Deduction in Multiple-Valued Logics*, volume 10 of *International Series of Monographs on Computer Science*. Oxford University Press, 1993.

18. Reiner Hähnle. Short normal forms for arbitrary finitely-valued logics. In *Proceedings ISMIS'93, Trondheim, Norway*, pages 49–58. Springer LNCS 689, 1993.
19. Reiner Hähnle. Efficient deduction in many-valued logics. In *Proc. International Symposium on Multiple-Valued Logics, ISMVL'94, Boston/MA, USA*, pages 240–249. IEEE Press, Los Alamitos, 1994.
20. Reiner Hähnle and Ortrun Ibens. Improving temporal logic tableaux using integer constraints. In *Proc. International Conference on Temporal Logic, Bonn, Germany*. Springer LNCS, 1994.
21. Reiner Hähnle and Werner Kernig. Verification of switch level designs with many-valued logic. In Andrei Voronkov, editor, *Proc. LPAR'93, St. Petersburg*, pages 158–169. Springer, LNAI 698, 1993.
22. J. N. Hooker and C. Fedjki. Branch-and-cut solution of inference problems in propositional logic. *Annals of Mathematics and Artificial Intelligence*, 1:123–139, 1990.
23. John N. Hooker. A quantitative approach to logical inference. *Decision Support Systems*, 4:45–69, 1988.
24. John N. Hooker. Logical inference and polyhedral projection. In *Proc. Computer Science Logic Workshop 1991, Berne*, pages 184–200. Springer, LNCS 626, 1991.
25. John N. Hooker. Generalized resolution for 0–1 linear inequalities. *Annals of Mathematics and Artificial Intelligence*, 6:271–286, 1992.
26. John N. Hooker. New methods for computing inferences in first order logic. Working Paper, GSIA, CMU Pittsburgh, April 1992.
27. R. Jeroslow. Computation-oriented reductions of predicate to propositional logic. *Decision Support Systems*, 4:183–187, 1988.
28. Robert G. Jeroslow. *Logic-Based Decision Support. Mixed Integer Model Formulation*. Elsevier, Amsterdam, 1988.
29. Robert G. Jeroslow and Jinchang Wang. Solving propositional satisfiability problems. *Annals of Mathematics and Artificial Intelligence*, 1:167–187, 1990.
30. Vadim Kagan, Anil Nerode, and V. S. Subrahmanian. Computing definite logic programs by partial instantiation and linear programming. Draft Manuscript, 1993.
31. Richard M. Karp. Reducibility among combinatorial problems. In R.E. Miller and J.W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
32. Richard C. T. Lee and Chin-Liang Chang. Some properties of fuzzy logic. *Information and Control*, 19(5):417–431, December 1971.
33. Shie-Jue Lee and David A. Plaisted. Eliminating duplication with the hyper-linking strategy. *Journal of Automated Reasoning*, 9(1):25–42, 1992.
34. Andreas Manuth. Vergleich von Normalformen zu Formeln in Łukasiewicz Logik. Studienarbeit, Universität Karlsruhe, Fakultät für Informatik, October 1993.
35. Robert McNaughton. A theorem about infinite-valued sentential logic. *Journal of Symbolic Logic*, 16(1):1–13, 1951.
36. Daniele Mundici. Satisfiability in many-valued sentential logic is NP-complete. *Theoretical Computer Science*, 52:145–153, 1987.
37. Daniele Mundici. The complexity of adaptive error-correcting codes. In *Proceedings Workshop Computer Science Logic 90, Heidelberg*, pages 300–307. Springer, LNCS 533, 1990.
38. Daniele Mundici. Normal forms in infinite-valued logic: The case of one variable. In *Proceedings Workshop Computer Science Logic 91, Berne*, pages 272–277. Springer, LNCS 626, 1991.
39. Daniele Mundici. A constructive proof of McNaughton's Theorem in infinite-valued logic. *Journal of Symbolic Logic*, to appear, 1994.

40. Daniele Mundici and Massimo Pasquetto. A proof of the completeness of the infinite-valued calculus of Łukasiewicz with one variable. In E. .P. Klement and U. Hoehle, editors, *Proc. International Conference on Nonclassical Logics and their Applications 1992 in Linz/Austria*. Kluwer, 1994.
41. David A. Plaisted and Shie-Jue Lee. Inference by clause matching. In Zbigniew Ras and Maria Zemankova, editors, *Intelligent Systems—State of the art and future directions*, chapter 8, pages 200–235. Ellis Horwood, 1990.
42. W. A. Pogorzelski. The deduction theorem for Łukasiewicz many-valued propositional calculi. *Studia Logica*, 15:7–23, 1964.
43. Helena Rasiowa. *An Algebraic Approach to Non-Classical Logics*, volume 78 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, 1974.
44. Klaus Ries and Reiner Hähnle. Prädikatenlogisches Beweisen mit gemischt ganzzahliger Optimierung. Ein tableaubasierter Ansatz. In *Working Notes of Workshop Künstliche Intelligenz und Operations Research, Berlin (published as Tech Report, Max-Planck-Institut für Informatik, Saarbrücken, MPI-I-93-???)*, 1993.
45. Harvey Salkin and Kamlesh Mathur. *Foundations of Integer Programming*. North-Holland, 1989.
46. Jörg Siekmann and Graham Wrightson, editors. *Automation of Reasoning: Classical Papers in Computational Logic 1967–1970*, volume 2. Springer-Verlag, 1983.
47. Raymond Smullyan. *First-Order Logic*. Springer, New York, 1968.
48. G. Tseitin. On the complexity of proofs in propositional logics. *Seminars in Mathematics*, 8, 1970. Reprinted in [46].
49. Ryszard Wójcicki. *Theory of Logical Calculi*. Reidel, Dordrecht, 1988.
50. Nicolas Zabel. *Nouvelles Techniques de Déduction Automatique en Logiques Polyvalentes Finies et Infinies du Premier Ordre*. PhD thesis, Institut National Polytechnique de Grenoble, April 1993.

View publication stats